



AI Code Generation

GitHub Copilot and Enterprise AI Tools

Chapter Overview

- 01** How AI Code Assistants Work
- 02** GitHub Copilot in Practice
- 03** Prompt Engineering for Developers
- 04** AI-Assisted Validation and Testing
- 05** Enterprise AI and Responsible Use



How AI Code Assistants Work

Section 1



The AI Revolution in Software Development

Large Language Models for Code

- Trained on large-scale mixtures of licensed, public, and synthetic code and text data
- Predict likely next tokens based on context, further refined through instruction tuning, reinforcement learning, and system-level integrations

What AI Coding Tools Can Do

- Code completion, function generation, and refactoring suggestions
- Bug detection, test generation, documentation, and code explanation

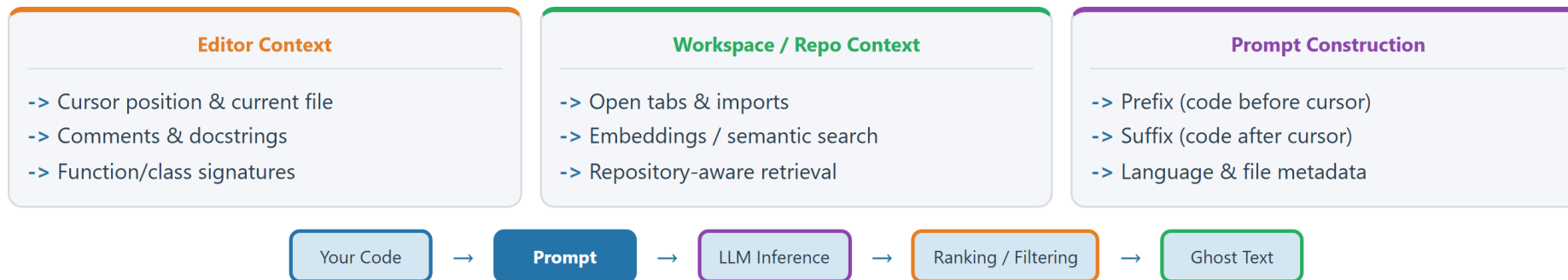
Key Principle

- AI amplifies developer productivity — it does not replace understanding



How Copilot Generates Suggestions

How Copilot Generates Suggestions



Context is assembled from editor, workspace, and repo signals • Post-processing ranks and filters before rendering ghost text

Context Is Everything

What Copilot Sees

- The current file — code before and after your cursor (prefix + suffix)
- Open tabs, imports, and increasingly the broader workspace (via semantic indexing and embeddings) provide additional signal
- Comments, function names, and type annotations guide generation

Context Window Limits

- Models have a finite context window, but modern tools augment this with multi-file context, semantic search, and repository-aware retrieval
- Code far from the cursor or in closed files may not influence suggestions

Hallucination Risk

- Models can confidently produce plausible but incorrect code — always verify
- Models may invent APIs or use outdated/deprecated versions of libraries



GitHub Copilot Overview

Copilot Interfaces

- Inline completions — real-time ghost text suggestions as you type
- Copilot Chat — conversational AI panel in the editor sidebar
- Copilot in the CLI — Copilot CLI capabilities vary by version and may change; refer to GitHub CLI documentation for current commands

Supported Environments

- VS Code, Visual Studio, JetBrains IDEs, and Neovim
- Works with JavaScript, TypeScript, Python, Java, Go, and most popular languages



GitHub Copilot in Practice

Section 2



Inline Code Completion

How It Works

- Copilot analyzes your current file, imports, and comments
- Displays one or more completions as dimmed ghost text at the cursor

Default VS Code Keybindings

- Tab — accept the current suggestion
- Esc — dismiss the suggestion
- Alt+] / Alt+[— cycle through alternative suggestions

Effective Workflow

- Write a clear comment or function name, pause, review the suggestion, then accept or edit



Maximizing Inline Suggestion Quality

High-Signal Comments

- Describe inputs, outputs, and edge cases in the comment before the function
- Include the framework and library context: 'Express middleware', 'React hook'

File Structure Matters

- Keep related code in the same file — Copilot uses nearby code as context
- Import statements signal which libraries are available

Accept, Inspect, Edit, Test

- Accept the suggestion, inspect it line by line, edit as needed, then run tests
- Treat every suggestion as a draft — not a finished product

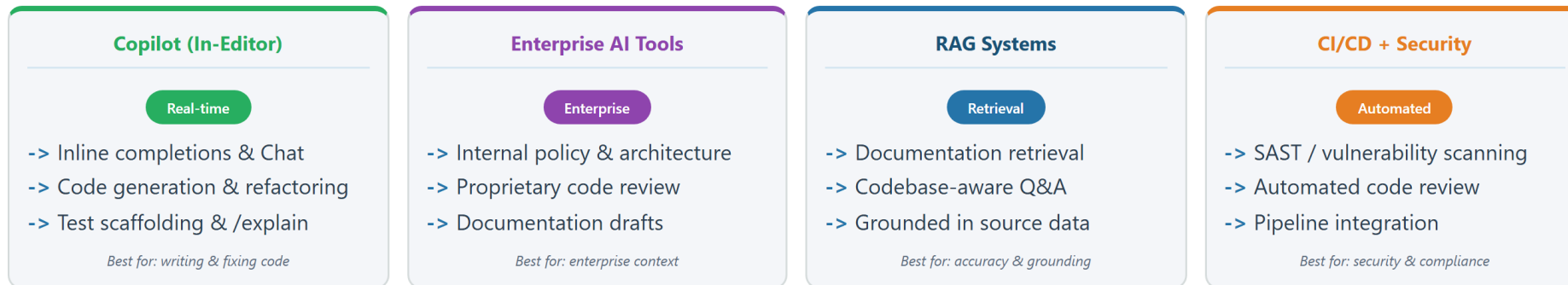


Comment-Driven Code Generation

```
// Validate a request payload for creating a new user.
// Requires: name (string, 1-100 chars), email (valid format),
//           age (integer, 18-120). Returns { valid, errors }.
function validateUserPayload(payload) {
  const errors = [];
  if (!payload.name || payload.name.length > 100)
    errors.push('Name is required (1-100 characters)');
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(payload.email))
    errors.push('Valid email is required');
  const age = Number(payload.age);
  if (!Number.isInteger(age) || age < 18 || age > 120)
    errors.push('Age must be an integer between 18 and 120');
  return { valid: errors.length === 0, errors };
}
// Copilot generated the full implementation from the comment spec.
// Results vary and require validation – always verify edge cases.
// Note: Email regex is simplified and may not cover all valid formats.
```

Choosing the Right AI Tool

Choosing the Right AI Tool



Use Copilot for code-level IDE tasks • Enterprise AI for internal context • RAG for grounded answers • CI/CD for automated security

Copilot Chat — Conversational AI in the Editor

Opening Chat

- Use the Command Palette (Ctrl+Shift+P) and search 'Copilot Chat'
- Keybindings may vary by platform, version, and user configuration

Slash Commands (VS Code)

- /explain — explain selected code in plain language
- /fix — suggest fixes for errors or warnings in selections
- /tests — generate initial test scaffolding — always expand coverage and verify correctness

Best Practice

- Select specific code first, then use a slash command for targeted results



Copilot Chat — Fix and Explain

```
// Original code with issues – select and use /fix in Chat:
async function fetchUsers() {
  const res = await fetch('/api/users');
  const data = await res.json();    // No status check!
  return data;                      // No error handling!
}

// Copilot Chat suggests this improved version:
async function fetchUsers() {
  const res = await fetch('/api/users');
  if (!res.ok) {
    throw new Error(`HTTP ${res.status}: ${res.statusText}`);
  }
  return res.json();
}
```



Generating Tests with Copilot Chat

```
// Select validateUserPayload, use /tests in Copilot Chat:
describe('validateUserPayload', () => {
  it('returns valid for correct input', () => {
    const result = validateUserPayload({
      name: 'Alice', email: 'alice@example.com', age: 30,
    });
    expect(result.valid).toBe(true);
    expect(result.errors).toHaveLength(0);
  });

  it('rejects missing name', () => {
    const result = validateUserPayload({
      name: '', email: 'a@b.com', age: 25,
    });
    expect(result.valid).toBe(false);
    expect(result.errors[0]).toMatch(/Name/);
  });
});
// Review generated tests – add edge cases the AI may have missed
// /tests generates initial test scaffolding – always expand coverage
```



Refactoring with Copilot Chat

```
// Select this function and ask Chat: 'Refactor to async/await'
function loadUserProfile(userId) {
  return fetch(`/api/users/${userId}`)
    .then(res => {
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return res.json();
    })
    .then(user => fetch(`/api/teams/${user.teamId}`))
    .then(res => res.json());
}
```

```
// Chat produces a cleaner async/await version:
async function loadUserProfile(userId) {
  const userRes = await fetch(`/api/users/${userId}`);
  if (!userRes.ok) throw new Error(`HTTP ${userRes.status}`);
  const user = await userRes.json();
  const teamRes = await fetch(`/api/teams/${user.teamId}`);
  if (!teamRes.ok) throw new Error(`HTTP ${teamRes.status}`);
  return teamRes.json();
}
```



Prompt Engineering for Developers

Section 3



What Makes a Good Prompt

The Four Components

- Context — language, framework, project purpose
- Task — what specifically to build or do
- Constraints — requirements, boundaries, error handling
- Format — expected output shape, style, library choices
- Examples — sample inputs/outputs to guide structure (few-shot prompting)

Common Anti-Patterns

- Too vague: 'make a function' — no context, no constraints
- Ambiguous requirements: AI fills in gaps with assumptions you may not want



Anatomy of an Effective Prompt

Anatomy of an Effective Prompt

Prompt Anatomy — From Vague to Effective

```
// ❌ Vague: "make a function"

// ✅ Effective prompt:
// Context: Express.js middleware for a REST API
// Task: Validate JWT from Authorization header
// Constraints: Return 401 if missing/invalid,
// attach decoded payload to req.user
// Format: async function, use jsonwebtoken library
```

Context

Language, framework,
project purpose

Task

What specifically
to build or do

Constraints

Requirements,
boundaries, limits

Format

Expected output
shape & style

Examples

Sample I/O for
few-shot prompting



Five Prompt Strategies

Comment-Driven

- Write a detailed specification as a comment — Copilot implements it

Example-Driven

- Provide one working example — Copilot generates similar functions following the pattern

Constraint-Driven

- Specify what the code must or must not do: 'no external dependencies', 'handle null input'

Persona-Driven

- 'Act as a senior developer reviewing this code for security vulnerabilities'
- Persona-driven prompting can help tone/style, but structure and constraints are more reliable for code generation

Iterative Refinement

- Start simple, then follow up: 'now add error handling', 'make it work with TypeScript'



Example-Driven Prompting in Practice

```
// Express route handlers for a REST API.  
// Pattern: validate input, call service, return response.  
  
// Example: GET /api/products/:id  
router.get('/api/products/:id', async (req, res, next) => {  
  try {  
    const product = await productService.getById(req.params.id);  
    if (!product) return res.status(404).json({ error: 'Not found' });  
    res.json(product);  
  } catch (err) { next(err); }  
});  
  
// Copilot now suggests handlers for POST, PUT, DELETE  
// following the same validate → service → response pattern
```

Constraint-Driven Prompting

```
// Create a debounce utility function.
// Constraints:
//   - No external dependencies (pure JavaScript)
//   - Must support cancellation via .cancel()
//   - Must preserve 'this' context when invoked
//   - TypeScript-compatible (use JSDoc types)

/** @param {Function} fn @param {number} delay */
function debounce(fn, delay) {
  let timerId;
  const debounced = function (...args) {
    clearTimeout(timerId);
    timerId = setTimeout(() => fn.apply(this, args), delay);
  };
  debounced.cancel = () => clearTimeout(timerId);
  return debounced;
}
```



AI-Assisted Validation and Testing

Section 4



Why Validation Matters

AI Models Hallucinate

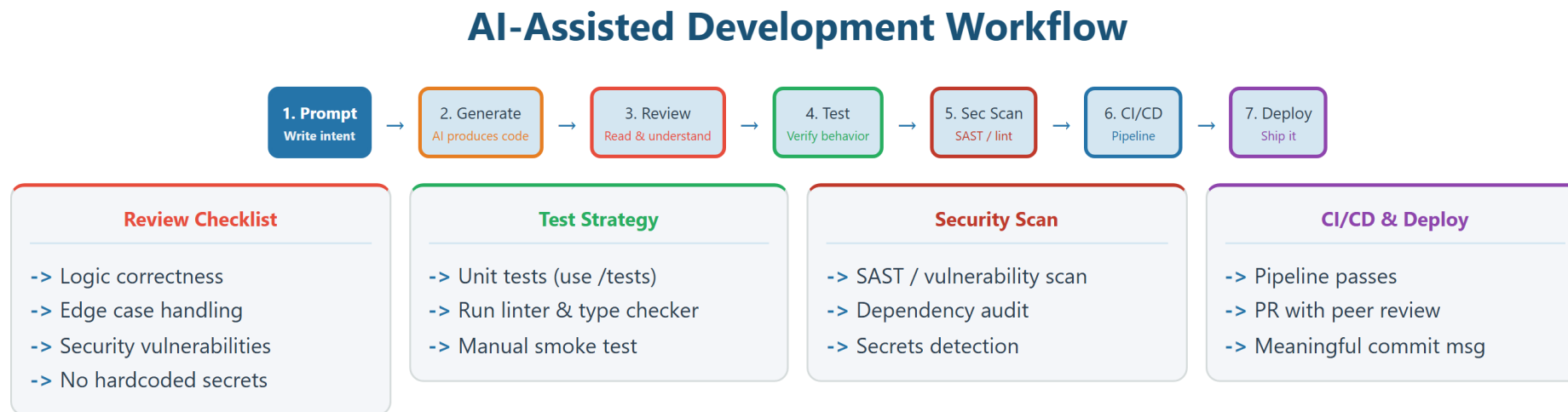
- Code can look correct, pass linting, yet contain subtle logic errors
- Models may invent API methods, parameters, or libraries that don't exist
- Outdated training data can produce deprecated patterns or insecure code

The Developer's Responsibility

- You own the code you commit — AI assistance doesn't shift accountability
- If you cannot explain what the code does, do not ship it



AI-Assisted Development Workflow



Validating AI-Generated Code

Read Before You Accept

- Understand the logic — trace through edge cases mentally
- Check for hardcoded values, missing error handling, and security issues

Automated Verification

- Run the linter and type checker — catch syntax and type errors immediately
- Execute existing tests to confirm the new code doesn't break anything
- Use static analysis / SAST tools to scan for known vulnerability patterns

Manual Smoke Testing

- Test the happy path and at least one failure case before committing



AI-Generated Code — Spot the Issues

```
// Copilot generated this API handler. Can you spot the problems?
app.post('/api/users', async (req, res) => {
  const user = req.body; // No validation!
  const result = await db.query(
    `INSERT INTO users (name, email) VALUES ('${user.name}', '${user.email}')`
  ); // SQL injection vulnerability!
  res.json({ id: result.insertId, ...user });
});

// Issues: no input validation, SQL injection via string interpolation,
// no error handling, no authentication check.
// AI often generates 'working' code that is insecure by default.
// Also vulnerable to logging sensitive data if request body is not sanitized.
// Always review for security before accepting suggestions.
```



Secure Version After Review

```
// After human review – parameterized query + validation
app.post('/api/users', authenticate, async (req, res, next) => {
  try {
    const { name, email } = req.body;
    if (!name || !email) {
      return res.status(400).json({ error: 'Name and email required' });
    }
    const result = await db.query(
      'INSERT INTO users (name, email) VALUES (?, ?)',
      [name, email] // Parameterized – no SQL injection
    );
    res.status(201).json({ id: result.insertId, name, email });
  } catch (err) {
    next(err); // Forward to error-handling middleware
  }
});
```



Enterprise AI and Responsible Use

Section 5



LibertyGPT for Enterprise Development

What is LibertyGPT?

- Liberty Mutual's internal AI assistant approved for enterprise workflows
- Example of an internal enterprise AI tool — equivalent tools vary by organization
- Safe for use with proprietary code, internal documentation, and architecture questions

When to Choose LibertyGPT over Copilot

- Questions involving internal systems, company architecture, or proprietary business logic
- Code review of internal services where context about company patterns is needed
- Drafting documentation, RFCs, or technical specs referencing internal systems

When to Use Copilot

- In-editor code completion, test generation, refactoring, and general coding tasks



Using LibertyGPT Effectively

Architecture and Design

- Ask for design pattern recommendations with company context included
- Get database schema feedback or API design reviews specific to your codebase

Code Review and Explanation

- Paste internal code and ask for review: performance, security, maintainability
- Get explanations of complex legacy code with full internal context

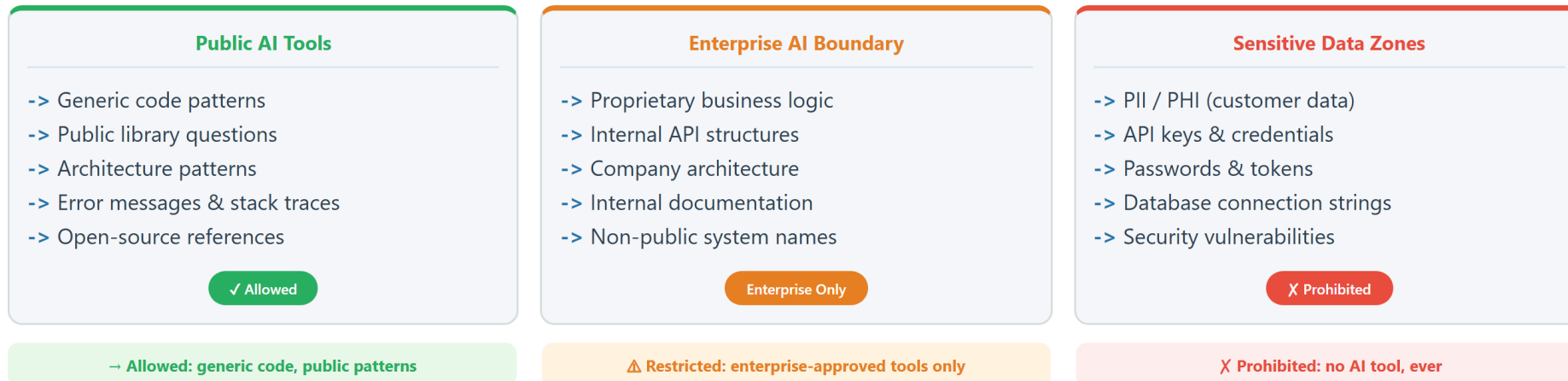
Safe Prompting for Enterprise

- Include relevant context about internal systems, but never include credentials or secrets
- Be specific — 'Review this Express middleware for our claims API' is better than 'review this code'



Enterprise AI Data Safety Boundaries

Enterprise AI Data Safety Boundaries



When in doubt, treat data as restricted • Follow your organization's AI usage policy



Enterprise AI Safety Guidelines

Data Classification Awareness

- Know your organization's data classification policy before using any AI tool
- LibertyGPT is approved for internal data — public AI tools may not be

Absolute Prohibitions

- Never paste API keys, passwords, tokens, or database credentials into any AI tool
- Never share customer PII, PHI, or other regulated data with AI assistants

Intellectual Property

- AI-generated code may require license review depending on company policy and tool settings
- Copilot Business/Enterprise may include IP protections depending on configuration and organizational policy



Building Good AI Habits

Always Review Generated Code

- AI produces plausible but potentially incorrect code — test everything
- Check for security vulnerabilities, edge cases, and logic errors

Understand What You Accept

- If you cannot explain the code, do not commit it
- Use AI output as a starting point — refine and customize for your needs

Maintain Your Skills

- AI is a productivity tool, not a replacement for fundamentals
- Practice solving problems without AI to keep your skills sharp



Common Pitfalls to Avoid

Over-Reliance

- Relying on AI for every line erodes problem-solving skills — stay engaged

Blind Acceptance

- Copilot suggestions are not always correct, secure, or performant
- Suggestion quality varies significantly based on prompt quality and context

Context Leakage

- Be mindful of what context you share — sensitive data, credentials, internal API details
- Treat proprietary source code as sensitive data unless explicitly approved

Skipping Tests

- AI-generated code still needs thorough testing — use /tests then review and augment



Security and Cost Considerations

Prompt Injection Risk

- Be aware of prompt injection risks when AI processes external or untrusted input

Cost and Latency

- AI-assisted workflows may introduce latency and cost — use selectively in performance-sensitive paths



Staying Current with AI Tools

Rapid Evolution

- AI coding tools ship new features and models frequently — capabilities change fast
- Model behavior can change across versions — validate outputs when models are upgraded
- Features like Copilot Chat commands may be added, renamed, or deprecated between versions

Keeping Up

- Follow the GitHub Copilot changelog and your IDE's release notes
- Experiment with new features in side projects before using them in production

Enterprise Considerations

- Your organization may control which features and models are available
- Check with your team lead before adopting new AI tool features on production code



Putting It All Together — Daily Scenarios

Building a New Feature

- Write a detailed comment spec → accept Copilot implementation → review → write tests with /tests

Debugging Production Code

- Select the error-producing function → use /explain to understand flow → use /fix to get suggestions

Onboarding to a Codebase

- Select unfamiliar functions and use /explain to learn the codebase faster
- Ask LibertyGPT about internal architecture decisions and team conventions

Writing Documentation

- Use Chat to generate JSDoc comments, README sections, and API docs from code



Modern AI Coding Capabilities

Beyond Simple Completion

- Modern AI coding tools may use agents, tool execution, and retrieval-augmented generation (RAG) to improve accuracy
- Agents can plan multi-step tasks, execute code, and iterate on results autonomously
- RAG systems retrieve relevant documentation and code to ground model responses



Key Takeaways

Key Takeaways

1. AI code assistants amplify productivity — they don't replace developer understanding
2. Copilot uses your editor context (file content, comments, imports) to generate suggestions
3. Good prompts include context, task, constraints, and expected format
4. Always validate AI-generated code: review, test, scan, then commit
5. Use LibertyGPT for enterprise questions — use Copilot for in-editor coding tasks
6. Never share credentials, PII, or secrets with any AI tool

